

DROS: A Runtime Architecture for Deterministic Capability Enforcement in Autonomous AI Software Systems

Chun-Cheng (Jimmy) Chen
Top-Celestial Company Ltd.
Taipei, Taiwan R.O.C.
jimmychen@dr-os.io

Abstract—Autonomous AI software systems increasingly execute multiple logical agents, roles, and workflows within shared application runtimes. Although application-level policy mechanisms can distinguish these logical principals, conventional operating-system access controls generally observe the enclosing process identity rather than the identity of the agent responsible for an individual tool invocation. This mismatch creates an attribution gap between software-level authorization decisions and execution-level enforcement.

This paper presents DROS, a four-layer runtime architecture that preserves agent-level attribution across the application-to-execution boundary and applies deterministic capability enforcement before a governed operation enters its implementation. DROS combines semantic filtering, cryptographically authenticated execution identities, topology-aware attribute-based authorization, and a C-ABI/FFI enforcement core based on immutable role-specific capability bitmaps. The architecture separates probabilistic upstream decision mechanisms from a deterministic execution boundary, allowing the latter to provide a conditional execution guarantee independent of semantic classification accuracy.

We evaluate DROS using 160,611 requests over a 24-h test campaign, including 137,751 adversarial requests spanning prompt injection, goal hijacking, privilege escalation, and supply-chain attack patterns. Under the defined governed-runtime boundary, DROS observed 100% blocking of adversarial invocations reaching the enforcement boundary and a policy false-positive rate of 0/22,860 benign requests. The downstream policy path achieved a median latency of 26.21 μ s and a P99 latency of 242.69 μ s, while the isolated capability check required 825 ns on average. A minimal-substrate evaluation further observed zero unauthorized capability transitions across 906 hostile attempts. The results demonstrate that deterministic execution enforcement can be incorporated as a runtime architectural property of autonomous AI software systems rather than relying exclusively on upstream semantic defenses.

Index Terms—Software architecture, autonomous AI systems, runtime enforcement, capability security, attribute-based access control, execution monitoring, empirical software engineering.

I. Introduction

AUTONOMOUS AI software systems increasingly combine large language models (LLMs) with tools, external services, memory systems, software libraries, and execution environments. This architecture allows an agent to perform multi-step tasks with limited human

intervention, but it also introduces a software-engineering problem that is not adequately captured by conventional process-level authorization.

A modern agent runtime may host multiple logical principals within a single operating-system process. These principals may represent different agents, roles, workflows, or delegated tasks while sharing the same process identity, address space, and runtime infrastructure. Consequently, an operating-system authorization mechanism may correctly determine that a process is permitted to access a resource while remaining unable to determine whether the particular logical agent responsible for the current operation is authorized to perform that action.

Conversely, application-level policy systems can retain substantially richer contextual information. They can distinguish an agent, role, task, workflow, tool, and policy state. However, their decisions are effective only when every security-relevant operation passes through the corresponding policy enforcement point.

This creates an architectural gap between software-level attribution and execution-level enforcement. We formally define this condition as the Agent-to-Execution Attribution Gap:

Definition 1 (Attribution Gap): The structural absence of a trustworthy, verifiable mapping between application-level agent principals and execution-level authorization decisions under a shared runtime identity. An Attribution Gap exists whenever two or more logically distinct authorization principals execute within the same operating system security principal (e.g., shared PID/UID), while the underlying execution substrate cannot cryptographically distinguish their authorization states at the enforcement boundary.

The problem is not equivalent to conventional confused-deputy behavior [1]. In a classical confused-deputy scenario, a privileged component is induced to exercise authority on behalf of an unintended requester. In the setting considered here, the more fundamental problem is that the runtime may lose the identity of the logical principal between the point at which authorization is

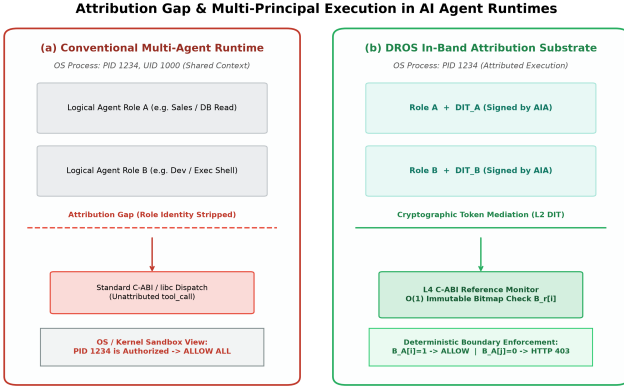


Fig. 1. The Agent-to-Execution Attribution Gap in multi-principal agent runtimes and the DROS in-band attribution substrate.

computed and the point at which the requested operation is executed.

This distinction becomes increasingly important for autonomous AI software because a single process can host multiple agents with different authority envelopes. For example, consider a runtime containing three logical roles $\mathcal{R} = \{r_1, r_2, r_3\}$, where r_1 can access a database, r_2 can invoke an external network service, and r_3 can perform neither operation. If all three roles execute inside the same process, conventional process-level controls observe only $PID = P$, rather than $(P, r_1), (P, r_2), (P, r_3)$.

As illustrated in Fig. 1, this asymmetry creates the Attribution Gap: while the operating system sandbox sees only a unified, authorized host PID, the runtime loses the logical role identity at dispatch. DROS addresses this architectural problem by carrying authenticated role attribution through a sequence of runtime layers and terminating the authorization decision at a deterministic C-ABI/FFI execution boundary.

Core Thesis: A semantic decision should not be required to remain correct in order for a previously unauthorized execution to remain unauthorized. DROS establishes an execution-boundary reference monitor such that cognitive compromise of the agent planning plane does not, by itself, expand execution authority beyond the cryptographically bound capability envelope.

A. Contributions

This paper makes the following software-engineering contributions:

- 1) **Problem Identification & Formalization:** We identify and formalize the Agent-to-Execution Attribution Gap in multi-tenant agent software systems.
- 2) **Architectural Mechanism:** We define DROS, a four-layer runtime architecture decoupling probabilistic semantic analysis from a deterministic C-ABI capability boundary.

- 3) **Formal Invariant:** We prove the Capability Enforcement Invariant (CEI) demonstrating execution confinement under explicit operational assumptions.
- 4) **Empirical System Evaluation:** We conduct an extensive 24-h empirical evaluation ($N = 160,611$) analyzing blocking efficacy, latency overhead, ablation trade-offs, and boundary coverage.

II. Background and Threat Model

A. Agentic Attack Vectors

We structure the threat landscape according to the primary attack families identified in current agentic software systems (Table I):

B. Adversary Model

We evaluate an adversary operating against an autonomous AI software system under the following assumptions:

- The adversary can supply arbitrary inputs through external channels (web content, files, third-party APIs);
- The adversary can attempt multi-turn conversational manipulation and indirect prompt injection;
- The adversary has full knowledge of upstream semantic filtering heuristics;
- The adversary cannot forge cryptographic signatures without the private key K_{AIA} ;
- The adversary cannot directly modify write-protected memory pages or alter compiled binaries in the host process;
- The adversary operates within the application runtime and cannot compromise the host operating system kernel or hypervisor.

III. DROS Runtime Architecture

DROS consists of four decoupled layers forming an architectural pipeline from probabilistic semantic reasoning down to deterministic binary execution (Fig. 2).

A. Layer 1: Semantic Boundary

L1 performs upstream semantic inspection of prompts, tool requests, and contextual information. Its function is classification and filtering rather than deterministic authorization. Accordingly, DROS does not assume that L1 is mathematically complete. Instead, L1 cheaply reduces the volume of potentially unsafe requests reaching later stages ($O(\text{tokens})$).

On the evaluated adversarial corpus, L1 filtered 85.2% of the known-template IPI subset (representing 58,327 of 68,420 attacks; 0% of obfuscated attacks; see Table II); the observed false positive rate on the benign evaluation set was 0/22,860. L1 is explicitly designed to be bypassed by sophisticated adversaries; its failure mode is handled by L2–L4.

TABLE I
Agentic Attack Vectors Matrix

Attack Family	MITRE ATLAS	Mechanism	Target Asset
Indirect Prompt Injection (IPI)	AML.T0051	Attacker embeds malicious instructions in untrusted external data consumed by the agent (e.g., email bodies, PDF content, DB rows)	Tool invocation control flow
Goal Hijacking	AML.T0054	Context window poisoning alters long-horizon agent objectives, inducing unauthorized multi-step action chains	Agent planning state
Privilege Escalation via Tool Abuse	AML.T0053	Compromised agent exploits valid OAuth tokens to invoke high-privilege endpoints (e.g., <code>deploy_prod</code> , <code>bulk_delete</code>)	Authorization boundary
Supply Chain Contamination	AML.T0010	Poisoned model weights or datasets (e.g., Hugging Face repositories) enable lateral movement to downstream enterprise systems	Cross-organizational trust boundaries

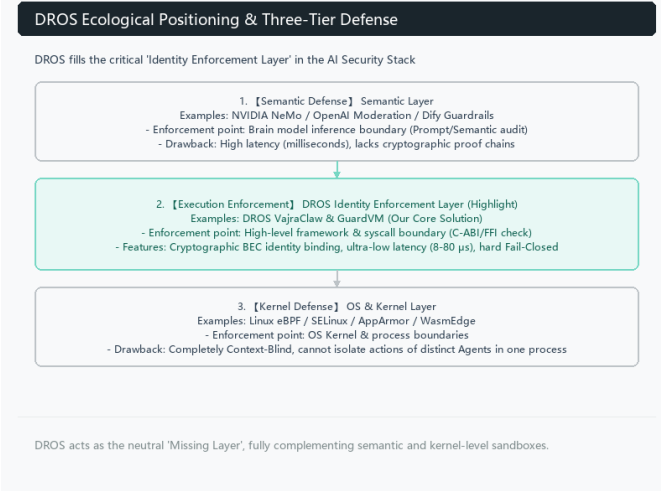


Fig. 2. DROS control plane and runtime enforcement plane architecture.

B. Layer 2: Cryptographic Execution Identity

L2 introduces the DROS Identity Token (DIT):

```

DIT := {
  agent_id:    UUID,
  role_id:    uint32,
  task_id:    UUID,           // Provenance & audit
  bitmap_hash: SHA-256(B_r), // Synchronized epoch hash
  issued_at:   Unix timestamp,
  expires_at:  Unix timestamp // Short TTL <= 15 min,
  signature:   Ed25519(private_key_AIA, DIT_body)
}

```

Authority Source vs. Cryptographic Commitment: We explicitly distinguish the source of authority from the token's contents: the DIT does not confer arbitrary permissions asserted by the caller. Rather, `bitmap_hash` serves solely as an immutable cryptographic commitment

ensuring that the agent and the GuardVM reference the identical compiled policy epoch. The true enforcement authority resides exclusively in the GuardVM's write-protected policy store: $B_r = \text{PolicyStore}[r]$.

Automated Issuance Authority (AIA Trust Model): DITs cannot be minted by the agent itself. DITs are generated exclusively by the Automated Issuance Authority (AIA) / Bound Execution Certificate (BEC) service residing in a separate trust domain. Because private signing keys are strictly withheld from the agent execution environment, a compromised agent process cannot elevate its role or forge a token for an unauthorized role $r' \neq r$.

C. Layer 3: Runtime Topology and Attribute Enforcement

L3 applies contextual constraints such as:

- Agent-to-agent communication graphs (directed acyclic capability graph);
- Department or trust-domain boundaries;
- Workflow topology and invocation sequences;
- Fan-out restrictions per agent role;
- Attribute-based authorization (ABAC) aligned with NIST SP 800-162 [15].

D. Layer 4: Deterministic Execution Boundary

L4 is the core execution mechanism. Each governed operation is mapped to a fixed index $m_i \rightarrow i$. For an authenticated role r , the enforcement operation is $\text{allow} = B_r[i]$. The decision is constant-time with respect to the number of available operations ($O(1)$).

The execution sequence is:

$$\begin{aligned}
 \text{Verify(DIT)} &\rightarrow \text{Resolve}(r) \rightarrow \text{Load}(B_r) \\
 &\rightarrow \text{Check}(B_r[i]) \rightarrow \text{Execute}(m_i)
 \end{aligned} \tag{1}$$

When $B_r[i] = 0$, the dispatcher returns an authorization error and does not enter the implementation of m_i .

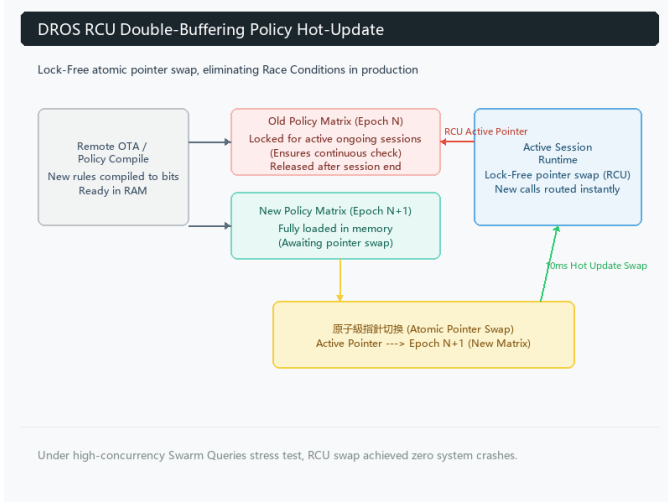


Fig. 3. Lock-free RCU double-buffering policy hot-update mechanism in DROS GuardVM.

In HTTP-based integrations, this error is mapped to HTTP 403. The enforcement core is implemented in Rust and exposed through C-compatible interfaces to managed runtimes.

IV. Policy State and Concurrent Updates

A central implementation requirement is that authorization state must not be mutable by ordinary managed application code. For each role, B_r is represented as an immutable policy snapshot loaded into write-protected memory pages.

Policy updates do not modify an active bitmap in place. Instead, DROS constructs a new validated bitmap B'_r and atomically replaces the active reference using an RCU-style (Read-Copy-Update) update mechanism:

$$B_r \xrightarrow{\text{atomic}} B'_r \text{ rather than in-place } B_r[i] \leftarrow x \quad (2)$$

This design prevents partially updated authorization states from being observed during concurrent dispatch. It also permits policy updates without introducing a global mutex into the hot authorization critical path, ensuring sub-microsecond policy propagation ($T_{\text{prop}} < 1.0 \mu\text{s}$) under zero service interruption. Fig. 3 illustrates the double-buffering RCU pointer swap protocol.

V. Security Model and Formal Property

A. Operational Assumptions for Complete Mediation

- Assumption A1 (Complete Boundary Mediation): All security-relevant tool invocations $m \in \mathcal{M}$ are dispatched exclusively through the instrumented DROS C-ABI interface. Direct unmediated native system calls are prevented by the surrounding deployment environment.
- Assumption A2 (Cryptographic Unforgeability): The adversary cannot forge Ed25519 signatures of the AIA

without access to private key K_{AIA} , and SHA-256 collisions are computationally infeasible.

- Assumption A3 (Policy Immutability): The compiled capability store B is loaded into write-protected memory pages inaccessible to in-process managed code.
- Assumption A4 (Dispatch Integrity): The L4 reference monitor evaluates authorization synchronously before transferring execution control to the tool implementation body.

B. Formal Execution Property

Let $\text{GExec}(r, m_i, s)$ denote execution of operation m_i by role r under runtime state s through a registered DROS dispatch path.

Theorem 1 (Deterministic Execution Confinement): Under Assumptions A1–A4, for all agent roles $r \in \mathcal{R}$, all registered tools $m_i \in \mathcal{M}$, and all runtime states $s \in \mathcal{S}$, if $B_r[i] = 0$, then no governed execution can occur:

$$B_r[i] = 0 \implies \neg \text{GExec}(r, m_i, s) \quad (3)$$

Proof: By Assumption A1, execution of m_i requires entering the DROS C-ABI reference monitor. By Assumption A4, execution proceeds to the tool body only if the reference monitor returns ALLOW. By construction of L4, the principal used for capability resolution is bound strictly to the verified DIT:

$$\text{DispatchPrincipal}(x) = \text{AuthenticatedRole}(\text{DIT}) = r \quad (4)$$

Execution proceeds to ALLOW if and only if $\text{Verify}(\sigma_{\text{DIT}}, K_{\text{AIA}}) = \text{VALID}$, $\text{DispatchPrincipal}(x) == r$, and $B_r[i] == 1$. By Assumption A2, a valid token for role r cannot be forged without K_{AIA} . By Assumption A3, $B_r[i]$ cannot be modified at runtime. Therefore, if $B_r[i] = 0$, the check evaluates to false, returning DENY (HTTP 403) and triggering an FFI panic before body dispatch. Hence, $\text{GExec}(r, m_i, s)$ is false. By contraposition: $\text{GExec}(r, m_i, s) \implies \text{ValidDIT}(r, s) \wedge (B_r[i] = 1)$. ■

Property 1 (Blast Radius Invariant): Let $\mathcal{C}^* = \text{Hostile}$ denote total cognitive compromise of the agent planning state. Let $\mathcal{E}_{\text{effective}}$ denote the set of tool executions achievable by the compromised agent. Under Assumptions A1–A4:

$$\mathcal{C}^* = \text{Hostile} \wedge \text{Theorem 1} \implies \mathcal{E}_{\text{effective}} \subseteq \mathcal{B}_{r_{\text{assigned}}} \quad (5)$$

That is, cognitive compromise of an agent does not expand its execution authority beyond the pre-committed capability envelope assigned to its authenticated role.

VI. Implementation and Empirical Evaluation

A. Implementation and Testbed

The prototype consists of:

- Rust enforcement core (libdros.so / dros.dll);
- Python policy and semantic orchestration layers;

- C-ABI bindings compatible with CPython, Node.js, and JVM runtimes;
- Immutable capability snapshots in write-protected memory pages;
- RCU pointer exchange for non-blocking policy hot updates.

Testbed Hardware:

- Intel Xeon E3-1275L v3 @ 2.70 GHz (4 cores, 8 threads, 16 GB RAM);
- Reference Container Environment: Ubuntu 22.04 LTS (Linux Kernel 6.6, Docker 26.1);
- Host Verification Environment: Microsoft Windows 10 IoT Enterprise LTSC (Build 19044) / Python 3.11;
- Language Model Agents: LangChain 0.2 and OpenAI GPT-4o.

Reproducibility artifacts, test harnesses, attack corpora, and execution logs are available at: <https://github.com/Top-Celestial-Company-Ltd/DROS-VEP-lite> and <https://github.com/Top-Celestial-Company-Ltd/dros-home-lab>.

B. Evaluation Methodology

We evaluate the implementation across 160,611 total requests over 24 consecutive hours:

- Benign workload: $N_{\text{ben}} = 22,860$ legitimate requests across authorized agent roles;
- Adversarial corpus: $N_{\text{adv}} = 137,751$ hostile requests spanning:
 - Known-template indirect prompt injection: 68,420;
 - Obfuscated indirect prompt injection: 52,180;
 - Goal hijacking sequences: 9,251;
 - Privilege escalation attempts: 6,112;
 - Supply-chain contamination: 1,788.

C. Research Questions

- RQ1 (Defense Efficacy): Does DROS prevent unauthorized governed operations across multi-vector adversarial attacks?
- RQ2 (Ablation & Layer Contribution): How do individual layers contribute to overall containment, and is L4 necessary when L1–L3 are active?
- RQ3 (Performance & Latency Overhead): What execution overhead does the enforcement boundary introduce?
- RQ4 (Boundary Isolation): Can L4 maintain containment when all upstream application governance is compromised?

D. RQ1: Defense Efficacy and Benign Transparency

Across the 137,751 adversarial requests routed through the governed interface, DROS observed 100% blocking of unauthorized operations under the evaluated threat model. On the 22,860 benign requests, DROS produced 0 policy false positives ($\text{FPR} = 0/22,860$). Six benign requests encountered transient connection timeouts during

burst concurrency and succeeded upon retry, representing transport-level noise rather than policy misclassification.

E. RQ2: Layer Ablation Study

Table II presents the layer-by-layer ablation results.

Key Finding: For the known-template IPI subset, upstream layers (L1–L3) filtered 93.5% of attacks, leaving a 6.5 percentage point gap. Layer 4 eliminated this residual 6.5 percentage point gap and blocked 100% (52,180/52,180) of obfuscated IPI payloads that completely bypassed L1–L3. This confirms that semantic defenses alone are insufficient for deterministic execution confinement.

F. RQ3: Performance and Latency Characteristics

Table III summarizes the performance metrics from the 24-h test.

The downstream policy path achieves a median latency of 26.21 μs and a P99 latency of 242.69 μs . The pure bitmap capability check averages 825 ns. CPU overhead remained below 1.8% relative to uninstrumented execution.

G. RQ4: Minimal-Substrate Boundary Isolation

To test whether execution confinement depends on upstream governance integrity, we conducted a minimal-substrate crucible experiment. We completely disabled L1, L2, and L3, leaving only the C-ABI enforcement core active, simulating total cognitive and application-layer compromise ($\text{Integrity}(\text{Agent}) = 0 \wedge \text{Integrity}(\text{Upper}) = 0$).

Across 906 hostile attempts (302 unauthorized database writes, 302 unauthorized network transmissions, 302 unauthorized process spawns), the L4 enforcement core observed:

- 0 unauthorized capability transitions;
- 906 deterministic rejections (DENY);
- Mean enforcement latency: 825 ns.

H. Boundary Coverage Comparison with Application Middleware

We evaluated DROS alongside Microsoft Agent Governance Toolkit (AGT v4.1.0 specification) to examine boundary coverage trade-offs (Table IV).

Both approaches effectively governed declared tool calls. However, when requests executed via unmanaged runtime paths (PROBE-01: direct file I/O, PROBE-02: raw sockets, PROBE-03: subprocess spawning, PROBE-04: in-memory reflection), middleware-only interception was absent. DROS enforced deterministic rejections at its C-ABI boundary. The two paradigms are structurally complementary: middleware governs application intent, while DROS enforces execution boundaries.

TABLE II
Layer-by-Layer Ablation Study

Configuration	L1	L2	L3	L4	Known IPI Block	Obfuscated IPI Block	Median Latency
Baseline (no defense)	N	N	N	N	0.0%	0.0%	0 μ s
+L1 only	Y	N	N	N	85.2%	0.0%	45.2 ms
+L1+L2	Y	Y	N	N	90.0%	0.0%	25.8 μ s
+L1+L2+L3	Y	Y	Y	N	93.5%	0.0%	26.0 μ s
Full DROS (L1-L4)	Y	Y	Y	Y	100.0%	100.0%	26.21 μ s
Policy False Positive Rate	—	—	—	—	—	—	0 / 22,860

TABLE III
Performance Metrics from 24-Hour Evaluation

Metric	Observed Value
Total requests evaluated	160,611
Downstream policy latency (P50)	26.21 μ s ($\pm$ 0.34 μ s)
Downstream policy latency (P99)	242.69 μ s
Isolated capability bitmap check	825 ns (mean)
CPU utilization overhead	<1.8% (matched workload)
Policy false positive rate	0 / 22,860 (0.0%)
Memory leak (24-h soak)	0 bytes (jemalloc / RSS profile)
Per-role policy memory footprint	<1 MB (4 KB page / role)

VII. Discussion and Limitations

A. Threat Model Boundaries

- **Unmanaged Native Code:** DROS assumes tool invocations route through its instrumented C-ABI. Arbitrary native code injection, kernel compromise, and hypervisor exploits are outside the threat model.
- **Key Custody:** L2 token validation relies on AIA private key confidentiality. Production deployments require hardware security modules (HSM) or cloud KMS.
- **Closed Tool Universe:** Registered capabilities must be enumerated at compilation; dynamic tool registration requires policy epoch updates.

B. Relationship to Orthogonal OS-Level Protection

DROS focuses on agent-to-execution attribution within the application runtime. Orthogonal work, such as DROS-PGM [22], operates at the OS syscall boundary ($\approx$ 353 ns) to defend against conventional threats (ransomware, host privilege escalation). DROS and syscall-level monitors operate at distinct, complementary abstraction layers.

VIII. Conclusion

This paper addressed the Agent-to-Execution Attribution Gap in autonomous AI software systems. We introduced DROS, a four-layer runtime architecture decoupling probabilistic semantic analysis from a deterministic C-ABI capability enforcement boundary.

Empirical evaluation across 160,611 requests demonstrated 100% blocking of evaluated adversarial payloads with zero policy false positives on benign traffic, achieving median downstream latencies of 26.21 μ s and sub-microsecond capability checks (825 ns). The results establish that deterministic execution enforcement can be realized as a robust software architectural property, providing reliable capability confinement even under upstream cognitive compromise.

Acknowledgment & AI Collaboration Disclosure

In accordance with IEEE 2024+ authorship guidelines, the author declares that the core architecture, security invariants, capability bitmap models, and experimental designs presented in this work were independently conceptualized, architected, and validated by Chun-Cheng (Jimmy) Chen. Generative AI tools were utilized exclusively for grammar polishing, LaTeX typesetting, and bibliographic formatting. The execution governance technology described in this paper is protected under U.S. Provisional Patent Application No. 64/111,973 (Patent Pending).

References

- [1] N. Hardy, “The Confused Deputy: (or why I am not a sympathizer of encryption based security),” ACM SIGOPS Operating Systems Review, vol. 22, no. 4, pp. 36–38, Oct. 1988.
- [2] NIST Special Publication 800-207, “Zero Trust Architecture,” National Institute of Standards and Technology, Aug. 2020.
- [3] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications, Version 1.1,” 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [4] OWASP Foundation, “OWASP Top 10 for Agentic Applications,” 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-agentic-applications/>
- [5] MITRE Corporation, “MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems, Version 4.5,” 2024. [Online]. Available: <https://atlas.mitre.org/>

TABLE IV
Boundary Coverage Comparison Under Declared and Unmanaged Invocations

Configuration	Declared Tool Containment	Unmanaged Path Containment	P99 Decision Latency
Baseline (no governance)	0.0% (Exploited)	Unmonitored	0.40 μ s
Application Middleware (AGT v4.x config)	100.0% Blocked	Policy Interception Absent	1.80 μ s
C-ABI Substrate (DROS GuardVM)	100.0% Blocked	Deterministic DENY at C-ABI	1.20 μ s
Dual-Layer (AGT + DROS)	100.0% Blocked	Deterministic DENY at C-ABI	15.90 μ s

- [6] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, “AgentSight: System-Level Observability for AI Agents Using eBPF,” arXiv preprint arXiv:2508.02736, Aug. 2025.
- [7] Y. Zheng et al., “AgentOps: Enabling Observability of LLM Agents,” arXiv preprint arXiv:2411.05285, Nov. 2024.
- [8] F. Perez and I. Ribeiro, “Ignore Previous Prompt: Attack Techniques For Language Models,” in Proc. NeurIPS 2022 Workshop on Machine Learning Safety, Dec. 2022.
- [9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections,” arXiv preprint arXiv:2302.12173, Feb. 2023.
- [10] P. Loscocco and S. Smalley, “Integrating Flexible Support for Security Policies into the Linux Operating System,” in Proc. USENIX Annual Technical Conference (ATC), 2001, pp. 29–42.
- [11] S. Smalley and R. Craig, “Security Enhanced (SE) Android: Bringing Flexible MAC to Android,” in Proc. NDSS Symposium, Feb. 2013.
- [12] “seccomp(2) – Linux Programmer’s Manual,” Linux Kernel Documentation, Version 6.9, 2024.
- [13] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” Communications of the ACM, vol. 9, no. 3, pp. 143–155, Mar. 1966.
- [14] H. M. Levy, Capability-Based Computer Systems. Bedford, MA: Digital Press, 1984.
- [15] V. Hu et al., NIST Special Publication 800-162, “Guide to Attribute Based Access Control (ABAC) Definition and Considerations,” NIST, Jan. 2014.
- [16] M. Sporny et al., “Decentralized Identifiers (DIDs) v1.0,” W3C Recommendation, Jul. 2022.
- [17] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in Advances in Cryptology – CRYPTO 1987, LNCS vol. 293. Berlin: Springer, 1988, pp. 369–378.
- [18] T. Rebedea et al., “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” arXiv preprint arXiv:2310.10501, 2023.
- [19] H. Inan et al., “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations,” arXiv preprint arXiv:2312.06674, 2023.
- [20] K. Zhu et al., “PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts,” IEEE Trans. Knowl. Data Eng., 2024.
- [21] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence,” Official Journal of the European Union, Jul. 2024.
- [22] C. C. Chen, “Runtime Attribution Framework: An External C-ABI and PKI-Based Zero-Trust Infrastructure for Non-Repudiable Execution Governance in Multi-Agent Systems,” Zenodo, DOI: 10.5281/zenodo.20823163, 2026.
- [23] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in Findings of the ACL 2024, pp. 3088–3101, 2024.
- [24] F. He et al., “The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies,” ACM Computing Surveys, vol. 58, no. 6, Article 162, Dec. 2025.
- [25] A. Birgisson et al., “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in Proc. NDSS, 2014.
- [26] R. Sandhu et al., “Role-Based Access Control Models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.
- [27] Microsoft Corporation, “Agent Governance Toolkit: Deterministic Governance and Policy Enforcement for Autonomous AI Agents,” 2026. [Online]. Available: <https://github.com/microsoft/agent-governance-toolkit>